Көпағынды бағдарламалау

Thread және Runnable интерфейстерімен жұмыс істеу негіздері

СРО Проекті • Java Programming

Теориялық Сұрақтар

Көпағындылықтың негізгі ұғымдары мен анықтамалары

Негізгі Сұрақтарға Жауаптар

Thread деген не?

Thread (Ағын) — бұл бағдарламаның ішіндегі ең кіші орындалу бірлігі. Ол процестің ішінде бір уақытта бірнеше іс-әрекетті орындауға мүмкіндік береді.

Runnable не үшін қажет?

Runnable интерфейсі класс басқа кластан мұрагерлік алған жағдайда қолданылады. Бұл Java-дағы "көптік мұрагерлік" шектеуін айналып өтуге көмектеседі.

Артықшылықтары

Процессор ресурстарын тиімді пайдалану, бағдарламаның жауап беру жылдамдығын арттыру және күрделі есептеулерді параллель орындау.

Процесс Vs Ағын (Thread)

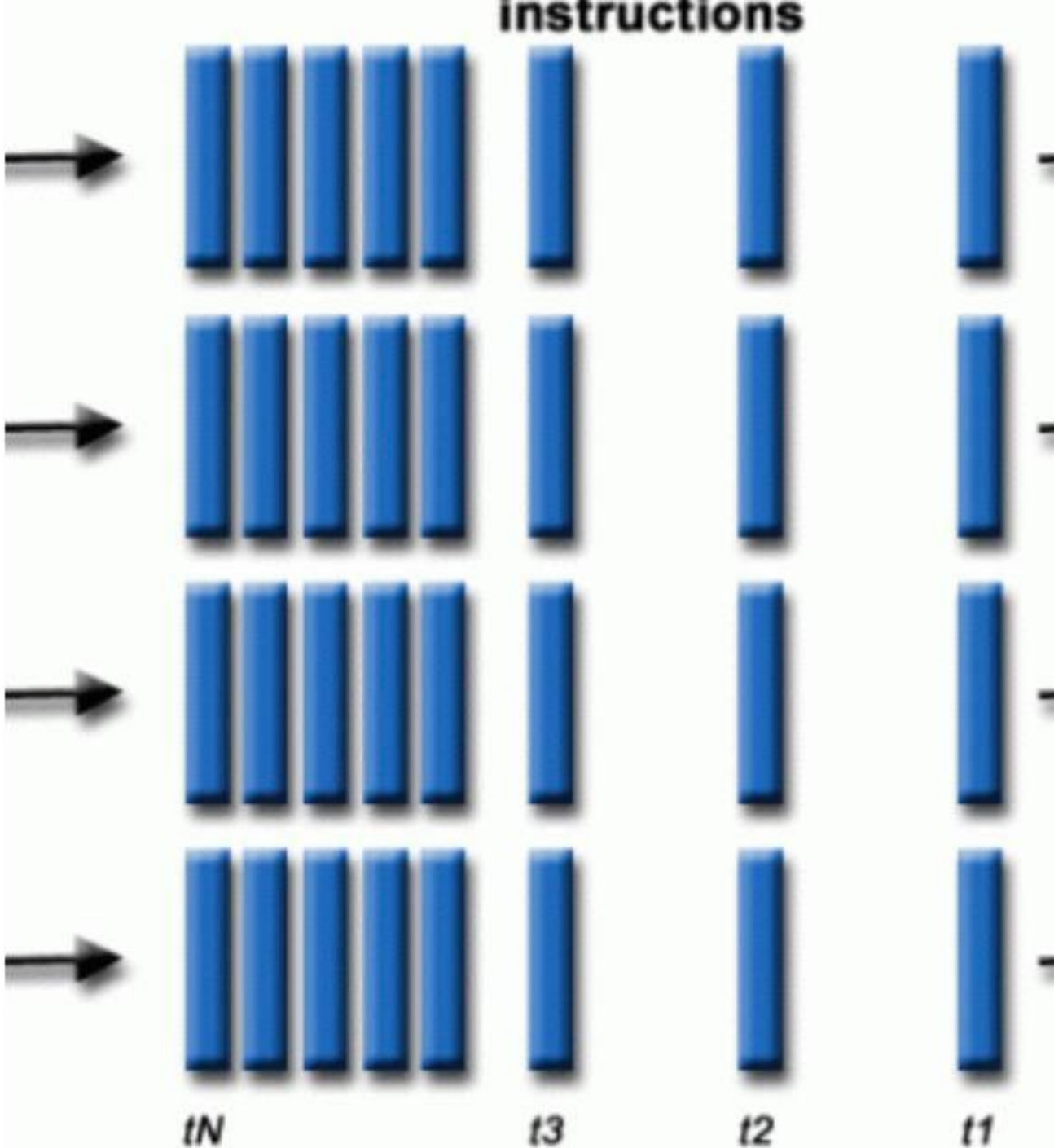
Процесс: Бұл орындалып жатқан бағдарлама, оның жеке жады бар.

Ағын: Бір процестің бөлігі, олар жалпы жадты бөліседі.

Ағындар арасында контекст ауыстыру процестерге қарағанда жылдамырақ.

Параллельді Орындау

Заманауи бағдарламаларда көпағындылық файлдарды фондық режимде жүктеу немесе күрделі математикалық есептеулерді пайдаланушы интерфейсін блоктамай орындау үшін қолданылады.



Көпағындылықтың Маңызы

- **Өнімділік:** Бірнеше ядролы процессорлардың әлеуетін толық пайдаланады.
- **Жауап беру:** Ауыр тапсырмалар орындалып жатқанда интерфейс қатып қалмайды.
- **Құрылымдылық:** Тапсырмаларды бөлек ағындарға бөлу бағдарлама кодын реттейді.
- **Серверлік шешімдер:** Бір уақытта мыңдаған сұраныстарды өңдеуге мүмкіндік береді.

Нақты Мысалдар

Файл жүктеу

Интернеттен үлкен файлды жүктеп жатқанда, бағдарламаның басқа батырмалары жұмыс істей береді. Бұл жүктеу ағынының бөлек болуының арқасы.

Есептеулер

Мәліметтер базасымен жұмыс немесе видео рендеринг жасағанда, есептеулер параллель жүреді, бұл уақытты 2–3 есе қысқартады.

Өнімділікті Салыстыру (Уақыт)

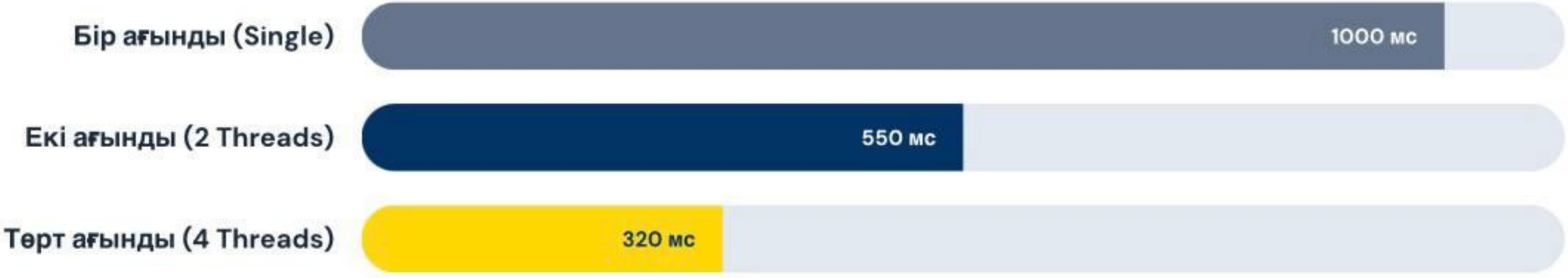


График ағындар саны артқан сайын тапсырманың орындалу уақытының қысқаратынын көрсетеді.

Thread Vs Runnable

Көрсеткіш	Thread класы	Runnable интерфейсі
Мұрагерлік	Басқа кластан мұрагерлік ала алмайды	Басқа кластан мұрагерлік ала алады
Икемділік	Төмен	Жоғары
Қолданылуы	Класс тек ағын болса тиімді	Кез келген логиканы ағынға қосу үшін
Ресурстар	Әр нысан жеке ағын	Бірнеше ағын бір нысанды бөлісе алады

Ағынның Өмірлік Циклі

Болашақ — Параллельдікте!

Көпағындылықты меңгеру — заманауи және жоғары өнімді бағдарламалық жасақтаманы жасаудың алғашқы қадамы.